

CUADERNOS DE COMPILADORES PARA APRENDIZAJE AUTOMÁTICO

Tutorial de CUDA

Del índice al kernel comprobado

Material para quien ha completado *Compiladores de ML desde cero*. Explicaciones paso a paso, programas completos, ejercicios con solución y validación local en CPU o GPU según la práctica.

Kripta Studios

Edición del 5 de octubre de 2026

Material didáctico independiente. Los programas originales y las evidencias acompañan al PDF en el repositorio. Las APIs externas se fijan a las versiones indicadas.

CAPÍTULO 1

Del compilador al programa CUDA

Ya has construido grafos, seguido índices y leído kernels en el libro de Lumbre. Ahora escribirás tú el programa que reserva memoria, copia datos, lanza un kernel y comprueba su salida. El objetivo es poder explicar cada acceso y cada espera antes de intentar acelerar una operación.

No necesitas conocer C++ avanzado. Usaremos funciones, punteros, bucles y una pequeña clase que libera memoria al salir de su ámbito. Introduciremos la sintaxis nueva cuando aparezca. Las prácticas trabajan con entradas deterministas y caben en la GPU del equipo de validación.

1.1 El recorrido y sus entregas

Práctica	Programa	Evidencia que producirás
Vectores	01_vectors.cu	Índices válidos, incluidos tamaños irregulares.
Reducción	02_reduce.cu	Suma por bloques y combinación en CPU.
GEMM	03_gemm.cu	Dos kernels con la misma salida y medidas repetidas.
Stream	04_stream.cu	Copia, kernel y copia ordenados en una cola.

Los archivos viven en `tutorials/cuda/`. El apéndice incorpora sus fuentes completos. Para trabajar, abre el archivo correspondiente, predice un resultado y ejecuta el programa. Conserva la versión correcta antes de modificarla.

Conectar con Lumbre

En Lumbre, el renderer produce el kernel y el runtime gestiona la memoria y el lanzamiento. Aquí separarás esas dos responsabilidades en el mismo archivo. Si la salida es incorrecta, podrás preguntar primero si falla el cálculo de índices o si el host entregó un buffer equivocado.

1.2 Qué se ha ejecutado en esta edición

La validación usa Ubuntu 24.04 bajo WSL2 y una NVIDIA GeForce RTX 5070 Ti Laptop, capacidad de cómputo 12.0. El compilador es `nvcc` 12.9.86; los programas se construyen para `sm_120`. El controlador observado es 591.86. El informe de cada programa conserva su hash y su código de salida.

Una arquitectura de compilación identifica las instrucciones que puede contener el binario. La cantidad de memoria, la potencia disponible y el tamaño de las matrices siguen condicionando la ejecución. El tutorial no necesita una GPU de centro de datos ni descarga modelos.

Antes de continuar

Localiza en el libro principal la diferencia entre forma y layout. Explica por qué una matriz de 17 por 19 no cabe en un único tile de 16 por 16. Si puedes dibujar los cuatro tiles y sus bordes, ya tienes la base para estas prácticas.

CAPÍTULO 2

Preparar el taller y entender la compilación

2.1 Tres piezas del entorno

El controlador permite que el proceso se comuniquen con el dispositivo. El toolkit proporciona `nvcc`, cabeceras y bibliotecas. El ejecutable resultante contiene código del host y código que se ejecutará en la GPU. Esas piezas tienen versiones distintas.

Abre una terminal de Linux o WSL y comprueba:

```
nvidia-smi
nvcc --version
```

La versión CUDA que aparece en `nvidia-smi` describe la compatibilidad del controlador; no demuestra que `nvcc` esté instalado. Si la segunda orden falla, localiza el toolkit y añade su carpeta `bin` a `PATH`. En este equipo se utiliza una instalación aislada de CUDA 12.9.1; en otro equipo la ruta puede ser distinta.

Desde la raíz del repositorio:

```
nvcc -O2 -lineinfo -std=c++17 -arch=sm_120 \
    tutorials/cuda/01_vectors.cu -o /tmp/cuda_vectors
/tmp/cuda_vectors
```

`-O2` solicita optimización, `-lineinfo` conserva información de líneas para diagnóstico y `-std=c++17` fija el dialecto del host. Cambia `sm_120` si tu GPU requiere otro destino que el toolkit admita. No elijas la arquitectura por el número comercial de la tarjeta.

2.2 Ejecutar la campaña completa

```
python3 tutorials/validate_cuda.py --arch sm_120 \
    --output tutorials/reports/mi_cuda
```

El ejecutor usa la biblioteca estándar de Python. Compila los cuatro archivos en una carpeta temporal, guarda las salidas de los programas y elimina sus binarios al terminar. Si falla una compilación o una comparación, termina con error. Las carpetas de referencia de la edición no deben reutilizarse para tus experimentos.

2.3 C++ que aparece en los ejemplos

`std::vector<float>` posee un array en memoria del host. `data()` entrega un puntero a sus elementos y `size()` su longitud. Un `float*` contiene una dirección, pero su tipo por sí solo no dice en qué dispositivo vive.

`DeviceBuffer<float>` es una clase del tutorial. Su constructor reserva memoria de dispositivo; su destructor la libera. El destructor se llama al salir del bloque delimitado por llaves. Se prohíbe copiar el objeto porque dos propietarios que liberasen el mismo puntero provocarían un error. Esta técnica se suele llamar RAII: ligar la duración de un recurso a la de un objeto.

`auto launch = [&]() { ... };` crea una función local que utiliza las variables del ámbito que la rodea. En la práctica de GEMM sirve para repetir exactamente el mismo lanzamiento al calentar y al medir. No compila un kernel nuevo en cada llamada.

CAPÍTULO 3

Una operación elemento a elemento completa

3.1 El contrato antes del kernel

Queremos calcular $z_i = 2x_i - y_i$ para índices de cero a $n - 1$. Los tres arrays tienen longitud n , sus elementos son FP32 y la salida tiene almacenamiento propio. El programa admite $n = 0$ como caso vacío y no lanza ningún kernel en ese caso.

El kernel recibe punteros que ya deben ser válidos en la GPU. `const float*` impide escribir a través del puntero de entrada dentro de esta función. La palabra `__global__` identifica una función que el host puede lanzar en el dispositivo.

```
__global__ void axpby(const float* x, const float* y,
                    float* z, int n) {
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if (i < n) z[i] = 2.0f*x[i] - y[i];
}
```

La expresión del índice combina el número del bloque, el tamaño del bloque y la posición del hilo. Con 128 hilos por bloque y $n = 257$, el host lanza tres bloques. Hay 384 hilos, pero solo 257 escriben resultados. El `if` protege también las dos lecturas. La guía de programación documenta la jerarquía de hilos. [1]

3.2 Seguir un elemento de principio a fin

En el host llenamos `x` e `y`, calculamos una referencia y reservamos tres buffers. Copiamos las entradas con `cudaMemcpyHostToDevice`. Después lanzamos el kernel y copiamos `z` al host con `cudaMemcpyDeviceToHost`. La comparación ocurre cuando el trabajo de dispositivo ha terminado.



Los argumentos entre «< y >>» son la configuración de lanzamiento. Los argumentos entre paréntesis pertenecen al kernel. Intercambiar un puntero de entrada con uno de salida puede compilar y producir una respuesta incorrecta; el compilador no conoce nuestro contrato matemático.

3.3 Dos momentos distintos para comprobar errores

`cudaGetLastError()` después del lanzamiento detecta errores que CUDA puede identificar al enviar el trabajo. `cudaDeviceSynchronize()` espera y puede comunicar un error ocurrido durante su ejecución. El helper `CUDA(...)` imprime la operación y la línea antes de terminar si el estado es incorrecto. [2]

No uses `printf` de miles de hilos como prueba principal. Una salida de depuración puede ayudar a seguir un caso pequeño, pero la comprobación numérica debe recorrer todo el resultado.

Predicción con 129 elementos

Calcula cuántos bloques se lanzan y cuántos hilos quedan fuera del dominio. Busca el último hilo que escribe. Solución: dos bloques, 256 hilos, 127 inactivos; el índice 128 corresponde al hilo cero del segundo bloque.

CAPÍTULO 4

Memoria, accesos y resultados numéricos

4.1 Un puntero no representa una copia automática

La variable `dx.ptr` está en el host, pero su valor identifica almacenamiento de dispositivo. Leer `dx.ptr[0]` desde C++ del host no equivale a leer el primer dato mediante CUDA. En estos programas usamos reservas y transferencias explícitas para que la propiedad de cada array sea visible.

No liberamos una entrada mientras un kernel pueda utilizarla. En la primera práctica la sincronización termina el trabajo antes de salir del ámbito. En la práctica de streams esperaremos a la cola correspondiente antes de leer o liberar la memoria fijada del host.

4.2 Accesos contiguos

Los hilos vecinos del kernel vectorial acceden a elementos vecinos. Ese patrón permite que el dispositivo agrupe accesos a memoria. Un stride grande puede obligar a transportar más segmentos para obtener la misma cantidad de datos útiles. Para estudiar este efecto necesitas un experimento con el mismo número de operaciones y una salida verificable.

La contigüidad es una propiedad de las direcciones generadas, no del nombre de una dimensión. En una matriz almacenada por filas, el índice `row*k+q` recorre una fila cuando cambia `q`. En `q*n+col`, mantener `q` y variar `col` recorre posiciones contiguas de B.

4.3 Errores que una tolerancia no debe ocultar

El comparador verifica que la salida sea finita y aplica:

$$|z_i - r_i| \leq \text{atol} + \text{rtol}|r_i|.$$

La tolerancia absoluta importa cerca de cero; la relativa escala con el tamaño del resultado. Las entradas vectoriales son múltiplos de potencias de dos, elegidos para facilitar el seguimiento manual. Una campaña posterior debe incluir otros valores y escalas.

Para una reducción larga, cambiar el orden de suma cambia el redondeo. Si acumulas una referencia en FP64 y el kernel en FP32, una discrepancia pequeña puede ser esperable. Antes de ampliar el margen, comprueba que no faltan elementos, que la salida no contiene NaN y que las diferencias crecen de una forma coherente con el problema.

Diseñar un fallo observable

En una copia de trabajo, cambia el límite a $i < n-1$. El caso de un elemento debería fallar. Explica por qué un conjunto de pruebas formado solo por ceros podría no detectar esa escritura ausente. Restaura el límite antes de medir.

CAPÍTULO 5

Cooperación dentro del bloque: reducir una suma

5.1 Dividir el problema

Una reducción combina muchos valores en uno. El programa `02_reduce.cu` asigna hasta 128 entradas a cada bloque. El bloque produce una suma parcial y el host combina esas sumas en FP64. Así evitamos introducir a la vez una segunda reducción GPU y la gestión de sus buffers.

La memoria `__shared__` pertenece al bloque. Otro bloque tiene su propio array `tile`, aunque la declaración sea la misma. Cada hilo escribe una posición; los hilos que quedan fuera de la entrada escriben cero, el elemento neutro de la suma.

5.2 Por qué la barrera está fuera del condicional

Todos los hilos del bloque llegan a la primera barrera después de cargar. Luego la mitad suma pares, una cuarta parte suma pares de parciales, y así hasta obtener un valor. Tras cada nivel, una barrera garantiza que las lecturas del siguiente nivel vean las escrituras anteriores.

```
tile[t] = i<n ? x[i] : 0.0f;
__syncthreads();
for (int stride=64; stride>0; stride/=2) {
    if (t<stride) tile[t] += tile[t+stride];
    __syncthreads();
}
```

El contrato de este kernel exige exactamente 128 hilos. La longitud del array y la secuencia de strides dependen de ese número. Cambiar el lanzamiento a 256 hilos sin cambiar el kernel escribiría fuera de `tile`.

Un retorno temprano de los hilos que no tienen entrada rompería la estructura de participación en las barreras. En una operación con cooperación, estar fuera del dominio de salida no significa que un hilo pueda abandonar el algoritmo.

5.3 Completar la reducción en el dispositivo

Para una segunda práctica, reserva un buffer para los parciales y vuelve a aplicar el mismo kernel hasta que quede un valor. En cada etapa, la longitud nueva es el número de bloques de la etapa anterior. Conserva ambas reservas hasta que termine la etapa que las usa y evita leer y escribir sobre regiones que se solapan.

El tamaño cero necesita una decisión del host: la suma vacía vale cero y no necesita una grid vacía. Un tamaño de un elemento sigue el mismo contrato que los demás.

Dibujar el árbol

Haz el dibujo para ocho entradas 1, 2, 3, 4, 5, 6, 7 y 8. Tras combinar posiciones separadas por cuatro obtienes 6, 8, 10 y 12; después 16 y 20; finalmente 36. Indica en qué puntos dos hilos podrían competir si quitaras una barrera.

CAPÍTULO 6

GEMM: de la fórmula al tile

6.1 La versión directa

Para A de forma [M,K] y B de forma [K,N], queremos C de forma [M,N]. Cada hilo calcula una posición de C. El eje x recorre columnas y el eje y filas. El bloque es de 16 por 16, por lo que contiene 256 hilos.

$$C_{r,c} = \sum_{q=0}^{K-1} A_{r,q} B_{q,c}.$$

La versión directa repite lecturas: los hilos que producen una misma fila de C necesitan elementos comunes de A. Los que producen una misma columna necesitan elementos comunes de B. Esa reutilización motiva el tile.

6.2 Cargar una pieza de cada entrada

El kernel compartido reserva dos arrays de 16 por 16. Cada hilo carga un elemento de A y uno de B, espera, y utiliza la fila de un tile y la columna del otro. Los 256 hilos cooperan para producir hasta 256 resultados.

Para el tile que comienza en `base`, el hilo `(tx,ty)` carga A en `(row,base+tx)` y B en `(base+ty,col)`. Si una coordenada cae fuera, escribe cero. La primera barrera publica ambas cargas. La segunda impide sobrescribir el tile mientras otro hilo lo sigue consumiendo.

El acumulador `sum` vive en cada hilo y persiste durante todas las piezas de K. La escritura final necesita máscara porque un bloque de borde también contiene hilos sin salida válida.

6.3 Los bordes son parte del algoritmo

En la forma [17,19,23], el programa necesita dos bloques en M, dos en N y dos piezas de K. La última pieza solo contiene siete valores de reducción útiles. Las posiciones restantes de la memoria compartida deben contener cero en esa iteración, no los valores de la anterior.

El tutorial prueba [1,1,1], [17,19,23], [31,47,65], [128,128,128] y [256,256,256]. Cada forma se ejecuta con los dos kernels y se compara con una referencia que acumula en doble precisión.

6.4 Qué se puede concluir de la comparación

Ambos programas calculan la misma GEMM, pero gestionan la memoria de manera distinta. El tile introduce cargas compartidas y dos barreras por iteración. En matrices pequeñas, ese

trabajo puede superar el ahorro. Para matrices mayores, influyen el tráfico, las instrucciones y la ocupación.

Una GEMM FP32 escrita con estos bucles no se convierte en una operación Tensor Core por tener una GPU reciente. La demostración WMMA del libro principal tiene un contrato diferente de tipos, layout y participación por warp. Integrar esa ruta exige otra prueba de corrección.

Cambiar una sola decisión

Compara la forma irregular $[31,47,65]$ con $[32,48,64]$. Explica qué máscaras siguen siendo necesarias y cuántos tiles de K ejecuta cada una. La segunda usa cuatro piezas completas; la primera necesita cinco. Mide ambas antes de atribuir toda diferencia a la irregularidad.

CAPÍTULO 7

Medir trabajo terminado

7.1 Definir el intervalo

El programa de GEMM reserva y copia antes de medir. Ejecuta tres calentamientos y después recoge treinta muestras. Cada muestra encierra cien lanzamientos entre dos eventos CUDA en la misma cola. Divide el intervalo entre cien y expresa el resultado en microsegundos por lanzamiento.

La muestra puede incluir huecos de envío entre kernels. Describe el tiempo de esa secuencia residente, no una latencia de instrucción ni el coste completo del proceso. La referencia CPU, las reservas y las transferencias iniciales quedan fuera.

```
CUDA(cudaEventRecord(start));  
for (int repeat=0; repeat<100; ++repeat) launch();  
CUDA(cudaEventRecord(stop));  
CUDA(cudaEventSynchronize(stop));  
CUDA(cudaEventElapsedTime(&ms,start,stop));
```

Esperar al evento de fin hace que el intervalo esté disponible. Ordenamos las treinta muestras y promediamos las dos centrales para obtener la mediana. El registro conserva también mínimo y máximo. Si la dispersión cambia mucho entre ejecuciones, investiga temperatura, potencia y otros procesos antes de defender una mejora.

7.2 No mezclar los relojes de los dos libros

Lumbre mide desde Python y sincroniza el runtime. Este tutorial usa eventos y agrupa cien lanzamientos. Sus tablas responden a preguntas diferentes. Aunque ambos programas calculen GEMM, dividir una latencia de un libro por otra del otro no produce una comparación controlada de compiladores.

Para medir extremo a extremo, inicia un reloj del host antes de la reserva o del trabajo que decidas incluir y deténlo después de una sincronización final. Escribe esa frontera en el informe. Conserva los resultados numéricos: un programa más rápido que omite parte de C no es una mejora.

Interpretar unidades

Si cien lanzamientos tardan 0,8 milisegundos, cada uno tarda 8 microsegundos en promedio dentro de ese lote. Para una GEMM $[M,N,K]$, el recuento habitual es aproximadamente $2MNK$ operaciones. No presentes ese cociente como utilización del pico sin fijar también la precisión y la ruta de instrucciones.

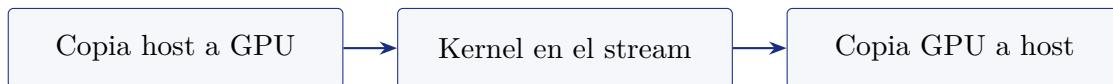
CAPÍTULO 8

Streams y vida de los buffers

8.1 Una cola con dependencias claras

En `04_stream.cu`, el host crea un stream, encola una transferencia de entrada, el kernel y la transferencia de salida. Las operaciones del mismo stream respetan ese orden. Antes de leer la salida, el host espera a ese stream.

Los arrays del host se reservan con `cudaMallocHost`: CUDA mantiene esas páginas fijadas para las transferencias. Se liberan con `cudaFreeHost`, después de la espera. No se debe reutilizar el array de entrada mientras la copia pendiente aún lo consume.



El cuarto argumento de lanzamiento especifica el stream. El tercero es la cantidad de memoria compartida dinámica y aquí vale cero. Las prácticas anteriores usan memoria compartida de tamaño conocido en compilación.

8.2 Asincronía y solapamiento

Una llamada asíncrona permite al host continuar antes de que termine el trabajo. El solapamiento exige además operaciones independientes y recursos disponibles. Este ejemplo utiliza una cola para enseñar dependencia y vida de buffers; no demuestra una aceleración mediante concurrencia.

Para extenderlo a dos trozos independientes, cada trozo necesitaría regiones separadas de entrada, salida y dispositivo. Una dependencia entre streams requiere una espera explícita, por ejemplo mediante un evento. La división de los datos no garantiza que la suma de los tiempos disminuya.

Encontrar la carrera del host

Describe qué puede pasar si lees `output[0]` antes de `cudaStreamSynchronize`. El host puede observar un valor anterior o todavía no escrito. Una ejecución que casualmente devuelve lo esperado no demuestra que el programa esté ordenado.

CAPÍTULO 9

Depurar, ampliar y volver al compilador

9.1 Un orden de investigación

Ante una salida incorrecta, reduce la forma, calcula una referencia y conserva el primer índice que difiere. Comprueba tamaños en bytes, orden de argumentos y máscaras. Después revisa barreras y duración de los buffers. Para una forma que solo falla en tiles incompletos, inspecciona primero las cargas enmascaradas.

Si dispones de Compute Sanitizer, puedes ejecutar una copia del binario con `compute-sanitizer -tool memcheck`. Las comprobaciones `racecheck` y `synccheck` estudian otros tipos de fallo. Esta edición no presenta esos análisis como ejecutados: la campaña incorporada compila, ejecuta y compara resultados.

Una prueba numérica y un analizador de memoria aportan evidencias distintas. Ni uno ni otro sustituye el razonamiento sobre el contrato y el dominio de índices.

9.2 Proyecto de cierre

Añade un bias por columna a GEMM y aplica ReLU antes de escribir C. Construye una referencia CPU que haga exactamente esa operación, incluyendo valores negativos. Conserva las cinco formas de prueba y mide la variante fusionada con la misma frontera que la original.

En una segunda versión, ejecuta GEMM y un kernel de bias/ReLU por separado. Compara resultados antes de comparar tiempos. Cuenta las escrituras y lecturas adicionales del intermedio. Este experimento conecta con la fusión del compilador sin requerir que cambies su IR.

9.3 Solución de diseño

En la variante fusionada, cada hilo calcula su acumulador, añade `bias[col]` y escribe el máximo con cero solo cuando su salida es válida. El bias tiene longitud N y debe copiarse al dispositivo. La versión separada recibe M por N elementos y obtiene la columna con el resto de dividir el índice lineal entre N.

La corrección incluye los bordes, la comparación con referencia y el rechazo de valores no finitos. El informe debe indicar si la medida incluye solo los kernels o también las transferencias. Conserva el código original para poder repetir la comparación.

Criterio de finalización

Has completado el tutorial cuando puedes compilar los cuatro programas, explicar los dos controles de error tras un lanzamiento, justificar ambas barreras de GEMM y distinguir

una cola asíncrona de un experimento que demuestra solapamiento. Entrega también una modificación con referencia y resultados, no solo una captura del tiempo.

Semanas 6–8: del rango del compilador al kernel

10.1 Conectar las prácticas con el syllabus

Compilers for Machine Learning introduce memoria, multihilo y rangos LOOP, UPGAST y THREAD en CPU durante la semana 6. Las semanas 7–8 añaden GPU, mapeo de ejes y tensor cores. Este capítulo, incorporado el 5 de octubre de 2026, propone ejercicios para integrar esas ideas en el compilador propio. Las mediciones del cuaderno conservan el alcance de su campaña original.

Los programas de vectores, reducción, GEMM y streams son referencias ejecutables. Los ejercicios siguientes son trabajo de ampliación, con soluciones razonadas; no se presentan como nuevas ejecuciones de tensor cores, HIP o comparaciones con PyTorch.

10.2 Un eje lógico y tres decisiones

Para $y[i]=(x[i]+2)*3$, el grafo puro contiene forma y operaciones aritméticas. Rangeify convierte la forma en rangos y accesos escalares. El planificador decide después qué trabajo es secuencial, se desenrolla o se distribuye. El renderer recibe esa decisión y sus límites; no debería inferir paralelismo a partir de nombres de variables.

Decisión	Pregunta antes de generar CUDA	Evidencia
Dominio	¿Qué posiciones debe producir el programa?	Intervalo lógico $[0,N)$
Reparto	¿Qué hilo calcula cada posición?	Fórmula de índice y cobertura
Dependencias	¿Qué debe terminar antes de usar cada dato?	Grafo de efectos y sincronización
Recursos	¿Qué almacenamiento y bloque requiere?	Validación frente al dispositivo

En CUDA, bloques e hilos identifican el trabajo y los hilos de un bloque pueden cooperar mediante memoria compartida y sincronización de bloque. Estas garantías no establecen un orden global entre bloques. Consulta el contrato del modelo de ejecución antes de escoger una bajada concreta. [1]

10.3 Ejercicio resuelto: 257 salidas

Una primera política usa un hilo por salida y 128 hilos por bloque. Para N positivo, la cantidad de bloques es el techo de N dividido entre 128. Para $N=257$ son tres bloques, 384 posiciones de hilo y 127 posiciones que no escriben. El índice es $\text{bloque} \cdot 128 + \text{hilo}$; la guarda de escritura es $i < N$.

Cobertura. Cada i válido tiene cociente y resto únicos al dividir por 128, que identifican su bloque e hilo. **Límites.** La guarda elimina índices entre 257 y 383. **Caso vacío.** Para $N=0$, el runtime puede omitir el lanzamiento y devolver la salida vacía según su contrato.

Compara kernel manual, C escalar y CUDA generado. Usa N igual a 0, 1, 127, 128, 129 y 257. Cambia los datos entre dos llamadas: acertar una única entrada no demuestra que el runtime actualice sus argumentos.

10.4 UPCAST no cambia la precisión

Supón que cada hilo calcula cuatro posiciones. Una política usa $i = \text{base} + \text{hilo} + q \cdot 128$, con q entre cero y tres y $\text{base} = \text{bloque} \cdot 512$. A q fijo, hilos consecutivos acceden a posiciones consecutivas. Otra asigna $i = 4 \cdot (\text{bloque} \cdot 128 + \text{hilo}) + q$; cubre el dominio, pero cambia el patrón simultáneo de accesos.

El upcasting puede exponer las cuatro operaciones como expresiones separadas y mantener varios acumuladores. Su beneficio depende de registros, instrucciones y memoria. Compara ambas políticas con el mismo contrato, sin asumir que cuatro elementos por hilo será siempre mejor.

Solución para $N=513$. En la primera política hacen falta dos bloques. El primero cubre 512 posiciones. En el segundo solo el hilo cero con $q=0$ produce un elemento válido. Cada escritura potencial necesita su propia condición; comprobar únicamente la base del hilo puede permitir accesos fuera de rango.

10.5 THREAD en un eje de reducción

Cambiar un rango de LOOP a THREAD no basta si todos los participantes actualizan el mismo acumulador. Diseña parciales privados, cooperación dentro del bloque y combinación de bloques. La práctica existente combina parciales en CPU; una extensión puede usar un segundo kernel.

Para $N=257$ y bloques de 128, la primera fase produce tres parciales y la segunda los reduce. Usa la identidad cero para posiciones enmascaradas y prueba valores positivos, negativos y cancelación. Si cambia el orden FP32, compara según la tolerancia del contrato.

Una barrera de bloque coordina producción y consumo dentro del bloque. Para participantes aún activos, ponerla en una condición que algunos cumplen y otros no puede dejar la sincronización sin un contrato válido. Los hilos de borde deben seguir la estrategia de participación y enmascarado prevista. [1]

10.6 STAGE, AFTER y vida de temporales

STAGE representa un temporal indexable. El plan debe identificar productor, consumidores, almacenamiento y última lectura. Una arena solo puede reutilizar el espacio cuando las dependencias lo permitan.

Imagina un productor P y consumidores Q y R. Que Q termine no permite sobrescribir el temporal mientras R siga pendiente. Una planificación secuencial en una cola puede establecer suficiente orden; varias colas necesitan dependencias explícitas entre ellas. La vida del objeto del host no determina la finalización de una lectura del dispositivo.

GROUP reúne resultados void sin ordenar sus fuentes; AFTER establece dependencia. Prueba dos salidas independientes y otra pareja donde la segunda consume lo escrito por la primera. El primer caso admite varios órdenes; el segundo exige observar el valor actualizado.

10.7 GEMM irregular: tres máscaras diferentes

Propón $M=17$, $N=19$, $K=23$ y tiles de lado 16. Hay cuatro tiles de salida y dos pasos por K. En el segundo paso solo siete posiciones de K son válidas. En la última fila de tiles existe una fila de salida; en la última columna, tres columnas.

Solución. Cada carga de A y B comprueba sus propias coordenadas y aporta cero fuera de dominio. La acumulación utiliza el tile con esos ceros; la escritura final comprueba M y N. Debe existir la sincronización requerida antes de consumir el tile y antes de sobrescribirlo en la siguiente iteración.

Reutilizar la misma condición para cargar A, cargar B y escribir C suele ser incorrecto porque sus límites difieren. Predice un elemento de borde con la referencia escalar y compara después toda la salida. Haz que el compilador conserve un dump del mapa de índices para esta prueba.

10.8 Tensor cores y selección por precondiciones

Una ruta matricial especializada exige tipos de entrada y acumulación, tiles, layouts, alineación y requisitos de participación de hilos. Los detalles dependen de la interfaz y arquitectura y deben consultarse antes de integrar el kernel. La selección comprueba esas condiciones y dispone de una alternativa correcta para los casos restantes. [1]

Como entrega, conserva el kernel general y añade una variante para un dominio explícito. Si necesita conversión, padding o reordenación, registra esos costes y su error numérico. No atribuyas una mejora exclusivamente a la instrucción cuando también cambia la precisión.

La demostración WMMA del libro principal no integra automáticamente tensor cores en el generador ni en el modelo. Acredita esa integración mostrando que una operación compilada selecciona la variante esperada y produce el resultado correcto.

CAPÍTULO 11

Semanas 9–11: medir el compilador dentro del modelo

11.1 Del tiempo de kernel al tiempo de entrenamiento

La ejecución residente mide trabajo con datos disponibles en dispositivo. La medida extremo a extremo incorpora preparación y transferencias pertinentes. El paso de entrenamiento incluye forward, backward y optimizador. Conserva esas fronteras separadas.

Si un kernel ocupa el 20 por ciento del paso y se acelera por dos sin cambiar nada más, el nuevo coste relativo es $0.8 + 0.2/2 = 0.9$: la aceleración ideal del paso es aproximadamente 1,11. La mejora del kernel no implica duplicar tokens por segundo. Este cálculo es una predicción; el experimento puede mostrar otras interacciones.

11.2 La meta de competir con PyTorch

El syllabus fija competitividad CUDA/HIP frente a PyTorch como objetivo. Compara mismas formas, entradas, dtypes, precisión de acumulación y función matemática. Declara versión, dispositivo, modo de ejecución y sincronización. Si una ruta incluye transferencias y otra no, informa métricas distintas.

Guarda muestras tras calentamiento, mediana y dispersión. Incluye formas del modelo y casos irregulares que activen kernels alternativos. Omitir las formas que fallan no establece cobertura. Los informes publicados de este cuaderno no contienen esa campaña: será evidencia adicional del estudiante.

11.3 Un paso y su memoria

Enumera pesos, activaciones, gradientes, estados del optimizador y temporales. Anota qué kernels los producen y consumen. Un tensor que el forward ya no necesita puede seguir siendo necesario para backward.

Al introducir recomputación o fusión, compara pérdida, gradientes y actualización con el plan anterior antes de medir. Para reanudar, conserva todos los estados necesarios del entrenamiento; descargar solo pesos no cumple ese contrato.

11.4 Proyecto y aceptación

Una ampliación acotada puede implementar reducción residente, upcasting, selección de GEMM o reutilización de temporales. El informe contiene hipótesis, precondiciones, pruebas,

medidas y casos desfavorables.

Hito	Evidencia
Semana 7	Mismo grafo correcto en CPU y GPU; índices y vida de buffers revisados
Semana 8	Ruta optimizada con dominio explícito y alternativa correcta
Semanas 9–10	Paso del modelo con valores, gradientes, memoria y tiempo comprobados
Semana 11 en adelante	Contribución activable y desactivable; reproducción y defensa

Los kernels manuales conservan su papel como referencias pequeñas. La entrega muestra además el programa generado, su plan y la relación entre cada decisión del compilador y la ejecución.

CAPÍTULO 12

Resultados de esta edición

12.1 Corrección observada

Los cuatro programas compilaron y terminaron con código cero: siete casos de vectores, siete de reducción, diez de GEMM y una cadena de stream. Los 25 casos pasaron sus comparaciones. La campaña secuencial y las versiones se conservan en `tutorials/reports/campaign.json`; los registros individuales están en `tutorials/reports/cuda/`.

12.2 Intervalos de GEMM

Cada fila resume treinta muestras de cien lanzamientos residentes, con eventos CUDA. Los tiempos excluyen reservas, copias iniciales y referencia CPU. Son observaciones de este portátil bajo WSL; no se fijaron frecuencias ni se aisló la carga del escritorio.

M,N,K	Variante	Mediana (us)	Mínimo (us)	Máximo (us)
1,1,1	directa	14.980	11.519	27.921
1,1,1	tile	13.452	10.796	22.863
17,19,23	directa	12.557	9.618	21.535
17,19,23	tile	11.317	9.132	22.561
31,47,65	directa	11.487	8.646	18.793
31,47,65	tile	10.077	7.796	17.468
128,128,128	directa	10.408	9.161	11.997
128,128,128	tile	11.221	9.634	25.703
256,256,256	directa	29.201	25.146	40.934
256,256,256	tile	26.356	21.336	33.560

La dispersión forma parte del resultado. Una diferencia pequeña entre medianas frente a ese intervalo no sostiene una afirmación general sobre qué kernel gana. Para investigar rendimiento, repite la campaña con carga controlada y conserva el mismo contrato de medición.

12.3 Límites y repetición

Las entradas elegidas facilitan verificar índices y sumas; no cubren todos los valores FP32. No se ejecutó Compute Sanitizer ni se midió solapamiento entre streams. Los tests acreditan estos programas y estos casos. Usa la orden del capítulo de instalación para crear tus propios registros.

CAPÍTULO A

Código completo de las prácticas

A.1 tutorials/cuda/common.cuh

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <vector>

inline void checked(cudaError_t status, const char* expr, int line) {
    if (status != cudaSuccess) {
        std::fprintf(stderr, "line %d: %s: %s\n", line, expr,
                        cudaGetErrorString(status));
        std::exit(1);
    }
}

#define CUDA(expr) checked((expr), #expr, __LINE__)

// Owning storage: copying an owner would cause a double free.
template<class T> struct DeviceBuffer {
    T* ptr = nullptr;
    explicit DeviceBuffer(size_t count) {
        if (count) CUDA(cudaMalloc(reinterpret_cast<void*>(&ptr), count*sizeof(T)));
    }
    DeviceBuffer(const DeviceBuffer&) = delete;
    DeviceBuffer& operator=(const DeviceBuffer&) = delete;
    ~DeviceBuffer() { if (ptr) CUDA(cudaFree(ptr)); }
};

inline double compare(const std::vector<float>& got,
                      const std::vector<float>& expected,
                      double atol=1e-5, double rtol=1e-5) {
    if (got.size() != expected.size()) std::exit(2);
    double worst = 0;
    for (size_t i=0; i<got.size(); ++i) {
        double error = std::abs(double(got[i])-expected[i]);
        if (!std::isfinite(got[i]) || error > atol+rtol*std::abs(expected[i])) {
            std::fprintf(stderr, "mismatch at %zu: %.9g vs %.9g\n",
                          i, got[i], expected[i]);
            std::exit(2);
        }
        worst = std::max(worst, error);
    }
    return worst;
}
```

A.2 tutorials/cuda/01_vectors.cu

```
#include "common.cuh"

__global__ void axpby(const float* x, const float* y, float* z, int n) {
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if (i < n) z[i] = 2.0f*x[i] - y[i];
}

int main() {
    cudaDeviceProp prop{};
    CUDA(cudaGetDeviceProperties(&prop, 0));
    std::printf("device=%s cc=%d.%d\n", prop.name, prop.major, prop.minor);
    for (int n : {0, 1, 127, 128, 129, 257, 65537}) {
        std::vector<float> x(n), y(n), z(n), ref(n);
        for (int i=0; i<n; ++i) {
            x[i] = float(i%31-15)/16; y[i] = float(i%17-8)/8;
            ref[i] = 2*x[i]-y[i];
        }
        DeviceBuffer<float> dx(n), dy(n), dz(n);
        if (n) {
            CUDA(cudaMemcpy(dx.ptr, x.data(), n*sizeof(float), cudaMemcpyHostToDevice));
            CUDA(cudaMemcpy(dy.ptr, y.data(), n*sizeof(float), cudaMemcpyHostToDevice));
            axpby<<<(n+127)/128, 128>>>(dx.ptr, dy.ptr, dz.ptr, n);
            CUDA(cudaGetLastError());
            CUDA(cudaDeviceSynchronize());
            CUDA(cudaMemcpy(z.data(), dz.ptr, n*sizeof(float), cudaMemcpyDeviceToHost));
        }
        std::printf("n=%d max_error=%.9g PASS\n", n, compare(z, ref));
    }
}
```

A.3 tutorials/cuda/02_reduce.cu

```
#include "common.cuh"

// Contract: launch with exactly 128 threads per block.
__global__ void partial_sum(const float* x, float* partial, int n) {
    __shared__ float tile[128];
    int t = threadIdx.x, i = blockIdx.x*blockDim.x+t;
    tile[t] = i<n ? x[i] : 0.0f;
    __syncthreads();
    for (int stride=64; stride>0; stride/=2) {
        if (t<stride) tile[t] += tile[t+stride];
        __syncthreads();
    }
    if (t==0) partial[blockIdx.x] = tile[0];
}

int main() {
    for (int n : {0, 1, 127, 128, 129, 257, 65537}) {
        int blocks=(n+127)/128;
        std::vector<float> x(n), partial(blocks);
        double ref=0, result=0;
```

```

    for (int i=0; i<n; ++i) { x[i]=float(i%19-9)/8; ref+=x[i]; }
    DeviceBuffer<float> dx(n), dp(blocks);
    if (n) {
        CUDA(cudaMemcpy(dx.ptr,x.data(),n*sizeof(float),cudaMemcpyHostToDevice));
        partial_sum<<<blocks,128>>>(dx.ptr,dp.ptr,n);
        CUDA(cudaGetLastError());
        CUDA(cudaDeviceSynchronize());
        CUDA(cudaMemcpy(partial.data(),dp.ptr,blocks*sizeof(float),cudaMemcpyDeviceToHost))
        ↵ ;
    }
    for (float value : partial) result+=value;
    if (!std::isfinite(result) || std::abs(result-ref)>1e-5) return 2;
    std::printf("n=%d sum=%.9g error=%.9g PASS\n",n,result,std::abs(result-ref));
}
}

```

A.4 tutorials/cuda/03_gemm.cu

```

#include "common.cuh"

__global__ void gemm_plain(const float* a,const float* b,float* c,int m,int n,int k) {
    int row=blockIdx.y*16+threadIdx.y, col=blockIdx.x*16+threadIdx.x;
    if (row<m && col<n) {
        float sum=0;
        for (int q=0;q<k;++q) sum+=a[row*k+q]*b[q*n+col];
        c[row*n+col]=sum;
    }
}

__global__ void gemm_tiled(const float* a,const float* b,float* c,int m,int n,int k) {
    __shared__ float as[16][16], bs[16][16];
    int tx=threadIdx.x, ty=threadIdx.y;
    int row=blockIdx.y*16+ty, col=blockIdx.x*16+tx;
    float sum=0;
    for (int base=0;base<k;base+=16) {
        as[ty][tx]=(row<m && base+tx<k) ? a[row*k+base+tx] : 0;
        bs[ty][tx]=(base+ty<k && col<n) ? b[(base+ty)*n+col] : 0;
        __syncthreads();
        for (int q=0;q<16;++q) sum+=as[ty][q]*bs[q][tx];
        __syncthreads();
    }
    if (row<m && col<n) c[row*n+col]=sum;
}

int main() {
    const int shapes[][3]={{1,1,1},{17,19,23},{31,47,65},{128,128,128},{256,256,256}};
    for (auto& shape:shapes) {
        int m=shape[0],n=shape[1],k=shape[2];
        std::vector<float> a(m*k),b(k*n),c(m*n),ref(m*n);
        for (int i=0;i<m*k;++i) a[i]=float(i%23-11)/16;
        for (int i=0;i<k*n;++i) b[i]=float(i%17-8)/8;
        for (int r=0;r<m;++r) for (int col=0;col<n;++col) {
            double sum=0;
            for (int q=0;q<k;++q) sum+=double(a[r*k+q])*b[q*n+col];
            ref[r*n+col]=float(sum);
        }
    }
}

```

```

}
DeviceBuffer<float> da(a.size()),db(b.size()),dc(c.size());
CUDA(cudaMemcpy(da.ptr,a.data(),a.size()*4,cudaMemcpyHostToDevice));
CUDA(cudaMemcpy(db.ptr,b.data(),b.size()*4,cudaMemcpyHostToDevice));
dim3 grid((n+15)/16,(m+15)/16),block(16,16);
cudaEvent_t start,stop;
CUDA(cudaEventCreate(&start)); CUDA(cudaEventCreate(&stop));
for (int tiled=0;tiled<2;++tiled) {
    auto launch=[&]() {
        if (tiled) gemm_tiled<<<grid,block>>>(da.ptr,db.ptr,dc.ptr,m,n,k);
        else gemm_plain<<<grid,block>>>(da.ptr,db.ptr,dc.ptr,m,n,k);
        CUDA(cudaGetLastError());
    };
    for (int i=0;i<3;++i) launch();
    CUDA(cudaDeviceSynchronize());
    std::vector<float> samples;
    for (int trial=0;trial<30;++trial) {
        CUDA(cudaEventRecord(start));
        for (int repeat=0;repeat<100;++repeat) launch();
        CUDA(cudaEventRecord(stop)); CUDA(cudaEventSynchronize(stop));
        float ms; CUDA(cudaEventElapsedTime(&ms,start,stop));
        samples.push_back(ms*1000/100);
    }
    CUDA(cudaMemcpy(c.data(),dc.ptr,c.size()*4,cudaMemcpyDeviceToHost));
    double error=compare(c,ref);
    std::sort(samples.begin(),samples.end());
    float median=(samples[14]+samples[15])/2;
    std::printf("m=%d n=%d k=%d tiled=%d median_us=%.6f min_us=%.6f max_us=%.6f\n",
        m,n,k,tiled,median,samples.front(),samples.back(),error);
}
CUDA(cudaEventDestroy(start)); CUDA(cudaEventDestroy(stop));
}
}

```

A.5 tutorials/cuda/04_stream.cu

```

#include "common.cuh"

__global__ void square(const float* x,float* y,int n) {
    int i=blockIdx.x*blockDim.x+threadIdx.x;
    if (i<n) y[i]=x[i]*x[i];
}

int main() {
    constexpr int n=65537;
    float *input=nullptr,*output=nullptr;
    CUDA(cudaMallocHost(reinterpret_cast<void*>(&input),n*sizeof(float)));
    CUDA(cudaMallocHost(reinterpret_cast<void*>(&output),n*sizeof(float)));
    std::vector<float> ref(n);
    for (int i=0;i<n;++i) { input[i]=float(i%31-15)/16; ref[i]=input[i]*input[i]; }
    DeviceBuffer<float> dx(n),dy(n);
    cudaStream_t stream; CUDA(cudaStreamCreateWithFlags(&stream,cudaStreamNonBlocking));
    CUDA(cudaMemcpyAsync(dx.ptr,input,n*sizeof(float),cudaMemcpyHostToDevice,stream));
    square<<<(n+127)/128,128,0,stream>>>(dx.ptr,dy.ptr,n);
}

```

```

CUDA(cudaGetLastError());
CUDA(cudaMemcpyAsync(output, dy.ptr, n*sizeof(float), cudaMemcpyDeviceToHost, stream));
CUDA(cudaStreamSynchronize(stream));
std::vector<float> got(output, output+n);
std::printf("n=%d stream_chain max_error=%.9g PASS\n", n, compare(got, ref));
CUDA(cudaStreamDestroy(stream));
CUDA(cudaFreeHost(input)); CUDA(cudaFreeHost(output));
}

```

A.6 tutorials/validate_cuda.py

```

"""Compile and run all original CUDA tutorial programs in temporary storage."""
import argparse
import hashlib
import json
from pathlib import Path
import shutil
import subprocess
import tempfile

root=Path(__file__).resolve().parent
parser=argparse.ArgumentParser()
parser.add_argument('--arch', default='sm_120')
parser.add_argument('--output', type=Path, default=root/'reports/cuda')
args=parser.parse_args()
nvcc=shutil.which('nvcc')
if not nvcc: raise SystemExit('nvcc must be on PATH')
args.output.mkdir(parents=True, exist_ok=True)
report={'arch':args.arch, 'nvcc':subprocess.check_output([nvcc, '--version'], text=True), '
↳ programs': []}
with tempfile.TemporaryDirectory(prefix='cuda-lessons-') as directory:
    for source in sorted((root/'cuda').glob('*.cu')):
        exe=Path(directory)/source.stem
        command=[nvcc, '-O2', '-lineinfo', '-std=c++17', f'-arch={args.arch}', str(source), '-
↳ o', str(exe)]
        subprocess.run(command, check=True)
        run=subprocess.run([str(exe)], text=True, stdout=subprocess.PIPE, stderr=subprocess.
↳ STDOUT)
        (args.output/(source.stem+'.log')).write_text(run.stdout, encoding='utf-8')
        if run.returncode: raise RuntimeError(run.stdout)
        report['programs'].append({'file':source.name, 'sha256':hashlib.sha256(source.
↳ read_bytes()).hexdigest(),
                                'returncode':run.returncode, 'passing_cases':run.
↳ stdout.count('PASS')})
        print(source.name, run.stdout.count('PASS'), 'cases PASS', flush=True)
report['temporary_binaries_removed']=not Path(directory).exists()
(args.output/'results.json').write_text(json.dumps(report, indent=2)+'\n', encoding='utf-8
↳ ')

```

CAPÍTULO B

Fuentes y alcance

Los programas y explicaciones de este cuaderno son originales. El código del apéndice coincide con los archivos ejecutables del repositorio. Las referencias siguientes permiten consultar el contrato de las APIs y ampliar las prácticas; los tiempos locales proceden de nuestros informes.

Bibliografía

- [1] NVIDIA. *CUDA C++ Programming Guide*, versión 12.9.1. Modelo de ejecución, jerarquía de hilos, memoria compartida y streams. <https://docs.nvidia.com/cuda/archive/12.9.1/cuda-c-programming-guide/index.html>.
- [2] NVIDIA. *CUDA Runtime API*, versión 12.9.1. Gestión de memoria, eventos, streams y errores. <https://docs.nvidia.com/cuda/archive/12.9.1/cuda-runtime-api/index.html>.
- [3] NVIDIA. *CUDA C++ Best Practices Guide*, versión 12.9.1. Medición, acceso a memoria y validación numérica. <https://docs.nvidia.com/cuda/archive/12.9.1/cuda-c-best-practices-guide/index.html>.
- [4] NVIDIA. *Compute Sanitizer Documentation*. Consulta adicional; el informe de esta edición no acredita ejecución de Sanitizer. <https://docs.nvidia.com/compute-sanitizer/ComputeSanitizer/index.html>.

Índice detallado

1.	Del compilador al programa CUDA	1
1.1.	El recorrido y sus entregas	1
1.2.	Qué se ha ejecutado en esta edición	1
2.	Preparar el taller y entender la compilación	3
2.1.	Tres piezas del entorno	3
2.2.	Ejecutar la campaña completa	3
2.3.	C++ que aparece en los ejemplos	4
3.	Una operación elemento a elemento completa	5
3.1.	El contrato antes del kernel	5
3.2.	Seguir un elemento de principio a fin	5
3.3.	Dos momentos distintos para comprobar errores	6
4.	Memoria, accesos y resultados numéricos	7
4.1.	Un puntero no representa una copia automática	7
4.2.	Accesos contiguos	7
4.3.	Errores que una tolerancia no debe ocultar	7
5.	Cooperación dentro del bloque: reducir una suma	9
5.1.	Dividir el problema	9
5.2.	Por qué la barrera está fuera del condicional	9
5.3.	Completar la reducción en el dispositivo	9
6.	GEMM: de la fórmula al tile	11
6.1.	La versión directa	11
6.2.	Cargar una pieza de cada entrada	11
6.3.	Los bordes son parte del algoritmo	11
6.4.	Qué se puede concluir de la comparación	11
7.	Medir trabajo terminado	13
7.1.	Definir el intervalo	13
7.2.	No mezclar los relojes de los dos libros	13
8.	Streams y vida de los buffers	14
8.1.	Una cola con dependencias claras	14
8.2.	Asincronía y solapamiento	14
9.	Depurar, ampliar y volver al compilador	15
9.1.	Un orden de investigación	15
9.2.	Proyecto de cierre	15
9.3.	Solución de diseño	15

10. Semanas 6–8: del rango del compilador al kernel	17
10.1. Conectar las prácticas con el syllabus	17
10.2. Un eje lógico y tres decisiones	17
10.3. Ejercicio resuelto: 257 salidas	18
10.4. UPGRADE no cambia la precisión	18
10.5. THREAD en un eje de reducción	18
10.6. STAGE, AFTER y vida de temporales	19
10.7. GEMM irregular: tres máscaras diferentes	19
10.8. Tensor cores y selección por precondiciones	19
11. Semanas 9–11: medir el compilador dentro del modelo	20
11.1. Del tiempo de kernel al tiempo de entrenamiento	20
11.2. La meta de competir con PyTorch	20
11.3. Un paso y su memoria	20
11.4. Proyecto y aceptación	20
12. Resultados de esta edición	22
12.1. Corrección observada	22
12.2. Intervalos de GEMM	22
12.3. Límites y repetición	22
A. Código completo de las prácticas	23
A.1. tutorials/cuda/common.cuh	23
A.2. tutorials/cuda/01_vectors.cu	24
A.3. tutorials/cuda/02_reduce.cu	24
A.4. tutorials/cuda/03_gemm.cu	25
A.5. tutorials/cuda/04_stream.cu	26
A.6. tutorials/validate_cuda.py	27
B. Fuentes y alcance	28